Lab 1 Report: App Stack

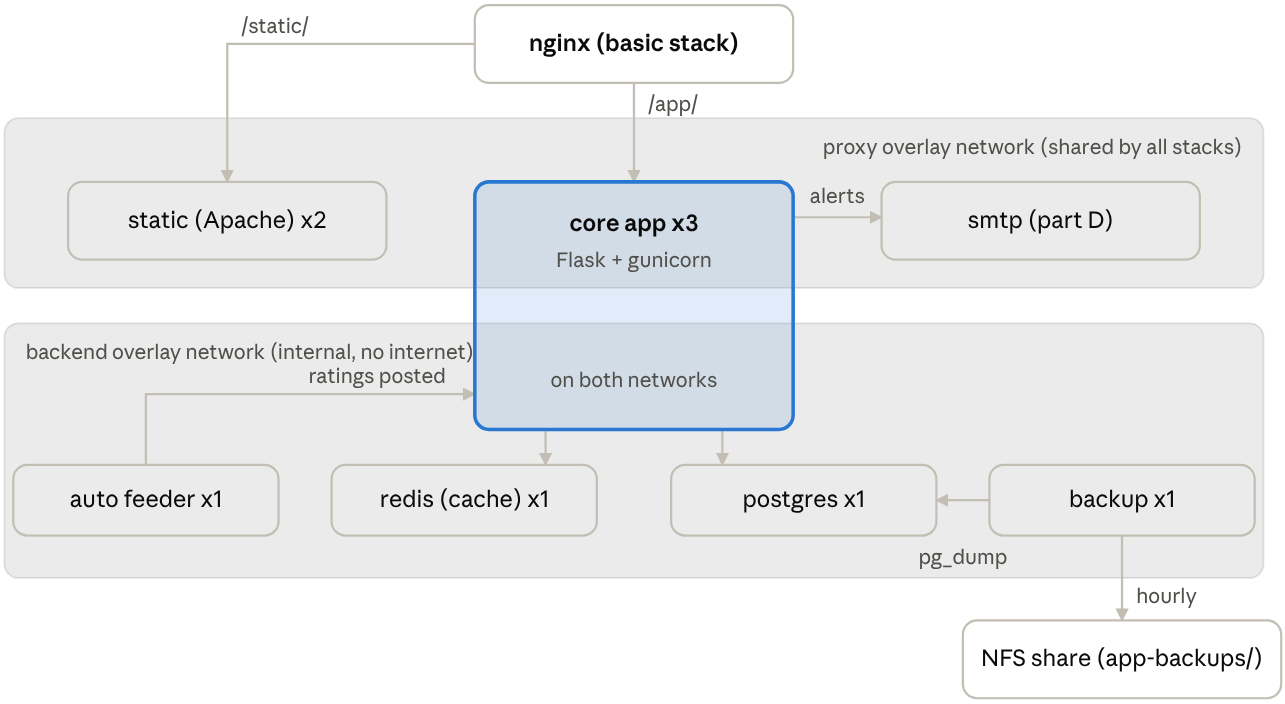
Overview

A small "page ratings" service is run by the app stack as six containers on Docker Swarm. Web pages are rated from 1 to 5 stars with a comment, either by visitors or by a bot, and a live leaderboard of the best rated pages is kept by the app. This idea was chosen because every component required by the assignment for the app stack is covered by it: a core app, a database, a cache, static files, an SMTP relay, an automatic feeder and a backup job.

Service	Image	Replicas	Networks	What it does
core	ccds/core-app:3 (Python, Flask, gunicorn)	3	proxy, backend	Ratings are stored, the leaderboard is served, alerts are sent and metrics are exposed
static	ccds/static:1 (Apache httpd)	2	proxy	The HTML page is served to users
auto	ccds/auto-feeder:1 (Python)	1	backend	A fake rating is posted every 5 seconds
db	postgres:17	1	backend	Every rating is stored (critical data)
redis	redis:7-alpine	1	backend	The leaderboard is cached for 30 seconds
backup	ccds/db-backup:2 (pg_dump)	1	backend	The database is dumped to NFS every hour

Five endpoints are exposed by the core app. A rating is saved through `POST /ratings`, the top ten pages are returned by `GET /leaderboard`, the status of the app and its dependencies is reported by `GET /health`, Prometheus counters are exposed for Grafana by `GET /metrics`, and the container hostname is returned by `GET /`, so the replica that answered can be identified. When a page is given a 1 star rating, an alert is emailed through the SMTP service of part D.

The data tier is placed on an internal network that cannot be reached by nginx



app stack architecture · 6 services, 2 networks

Only the core app and the static server are attached to the shared proxy network. Postgres, Redis, the feeder and the backup job are kept on the internal backend network, and the dumps are sent out of the cluster to the NFS share.

Design decisions

One process per container. A single foreground process is run in each container, and its logs are written to stdout, as recommended in the course. The backup job and the feeder were split into their own containers instead of being added to the app or the database. In this way, each one can be scaled, updated or restarted on its own.

Image builds. `requirements.txt` is copied and the Python packages are installed before the code is copied. Each layer is cached by Docker, so when `app.py` is edited, only the last layer is rebuilt, in a few seconds. Every package version is pinned, so the same image is built by every teammate. All images are run as a normal user instead of root, and the damage is limited if a container is broken into.

Versioned tags. The `latest` tag is never deployed. A version tag (`core-app:1` , `core-app:2` , `core-app:3`) is given to each image, and a new tag is created for every code change. In this way, rollouts and rollbacks are reduced to a one line change in the stack file.

Two networks. The database and Redis are placed only on a private `backend` overlay network, which is set to `internal` . No route to the internet is given to them, and they can be reached only by our own services. The core app and the static server are also attached to the shared `proxy` network, where they are reached by nginx from the basic stack as `app_core:8000` and `app_static:80` . The feeder is attached only to `backend` , because only the core app has to be reached by it.

Health check as a liveness check. In the first version, a failure was reported by `/health` whenever Postgres was down. It was realised that in Swarm every app replica would be killed at once and restarted in a loop, and one database problem would be turned into a full outage. A database problem cannot be fixed by restarting the app, so `/health` was changed: a 200 is returned while the Python process works, and the status of the database and Redis is only reported in the response. A 2 second database connection timeout was also added, so failed requests are ended quickly instead of being left hanging.

Graceful degradation. Email alerts are treated as best effort: if SMTP fails, the rating is still saved. If Redis fails, the cache is skipped and the leaderboard is read from Postgres. An essential component is never broken by a non essential one.

Data management

Everything stored by the stack was sorted into four kinds, following the questions raised in the volumes lecture: is it data at all, and if so, is it critical?

What	Kind	Where it lives	If it is lost
Ratings in Postgres	Critical, persistent data	Named volume `db_data` on the node labelled `db=true`	It is restored from the latest backup
Leaderboard in Redis	Volatile data (cache)	Kept in container memory only; disk writes are turned off	It is rebuilt from Postgres on the next request

What	Kind	Where it lives	If it is lost
HTML and CSS pages	Code, not data	Inside the `static` image	The image tag is redeployed
Database password	Secret	Docker secret, mounted by Swarm at `/run/secrets/db_password`	A new secret is created
Database dumps	Backup of critical data	NFS share, under `app-backups/`	The live database is still available

Why the database is kept off NFS. Databases on a network filesystem can be corrupted by file locking problems, and they are slowed down. For this reason, the Postgres files are kept on a local volume, and the `db` service is pinned to one node by a placement constraint (`node.labels.db == true`). Without this constraint, Postgres could be moved by Swarm to another node, where it would be started with an empty volume. The backups are sent to NFS instead, so they are preserved if the database node is lost.

Why no volume is given to Redis. Redis is started with `--save "" --appendonly no`, so nothing is ever written to disk. Its memory is also capped at 64 MB with the `allkeys-lru` policy, so the oldest keys are dropped when the cache is full, instead of the process being crashed. When the cache is lost, only speed is lost.

Secrets. The real password is stored in `secrets/db_password.txt`, which is kept out of the repository by `.gitignore`. A `db_password.txt.example` file is committed, so teammates are shown what has to be created. The secret is read by Postgres through `POSTGRES_PASSWORD_FILE`, and in the app and backup scripts a `_FILE` variable is checked first, with a plain environment variable used as a fallback. In this way, the same image can be run locally with a simple password and in production with a Docker secret.

Backups and disaster recovery

Only the critical data (the ratings) is backed up, and everything else is rebuilt from code, as suggested in the volumes lecture. The infrastructure is kept in Git as Dockerfiles and stack files, so after a disaster only the latest dump and the repository are needed.

The backup job. `backup.sh` is run in a loop by a dedicated `backup` container. Every hour, `pg_dump` is run against the database, its output is compressed by `pg_dump` itself, and the result is written to the NFS share. The dump is first written to a `.tmp` file, it is checked for the marker that `pg_dump` writes at the bottom of every complete dump, and only then is it renamed. If any step fails, the temporary file is deleted, the older backups are kept, and

the backup is retried after one minute instead of a full hour. After each run, all dumps except the newest 48 are deleted, so two days of history are kept without the disk being filled. The image is built from `postgres:17`, so the version of `pg_dump` is always matched to the server. The job is run as a single replica, since only duplicate dumps would be produced by two copies.

The restore script. A backup that is never restored is only a hope, so `restore.sh` was also written. The newest complete dump is selected (or a file name can be passed), and any incomplete dump is skipped. Before the database is touched, the login is tested. Only then is the schema dropped and recreated, and the dump is loaded with `ON_ERROR_STOP=1`, so the restore is aborted on the first error instead of a half restored database being left behind.

The recovery drill. The full cycle was tested locally with a 1 minute backup interval:

1. The feeder was left running until seven dumps were stored in the backup folder.
2. A disaster was simulated with `DROP TABLE ratings`.
3. `docker compose exec backup restore.sh` was run, and the 14:57:52 dump was loaded.
4. All pages were returned by the leaderboard again.

What was lost: the RPO. The data was brought back as of the last dump. Every rating that had been posted by the feeder between that dump and the `DROP TABLE` was lost for good. This gap is known as the Recovery Point Objective: the maximum data loss that is accepted is equal to the backup interval. In production, it was set to one hour. Following the categories given by Schmidt (2006), the ratings data is classed as business edge at most, so a one hour RPO is well within the one week limit for that category.

If the database node is lost. The Postgres files are kept on one node, so the live database is lost together with that node. The following plan was prepared: another node is labelled `db=true`, the stack is redeployed (Postgres is started there by Swarm with an empty volume), and `restore.sh` is run from the NFS backups.

What it costs in time: the RTO. After the two bugs described in Tests and results were fixed, the ratings table was dropped on the single node Swarm and `restore.sh` was timed: the data was brought back in 0.38 seconds, and the leaderboard was served again right after. Only the restore itself, on a small dataset, is covered by this number. For a full node failure, time is also needed to detect it, relabel a node and redeploy, so a real RTO of

a few minutes is expected, which is still far below the one week limit for business edge services. [The measured RTO for a full node failure will be added once the drill is run on the 3 node swarm.]

Swarm deployment

Two stack files are kept, as required by the assignment. With `compose.yml`, everything is run on one laptop with published ports and plain passwords for quick testing. The Swarm deployment is described in `production.yml`, and the differences are listed in the table below.

Setting	compose.yml (local)	production.yml (Swarm)
Published ports	8000 (core), 8080 (static)	None, nginx is the only way in
Passwords	Plain environment variables	Docker secret
Replicas	1 of each	core 3, static 2, others 1
Networks	Bridge, one machine	Overlay, spans all nodes
Backup storage	Local named volume	NFS share
Images	Built on the laptop	Pushed to the registry, so they can be pulled by every node

Replicas and placement. Three replicas of the core app are run with `max_replicas_per_node: 2`, so all of them are never stacked on one node. If a node fails, requests are still served by the other replicas while replacements are started elsewhere by Swarm. This is the reconciliation loop described in the orchestration lecture: the desired state is declared, and the real state is kept in line with it by Swarm.

Rolling updates and rollback. For the core app, `parallelism: 1`, `delay: 10s` and `order: start-first` were set. One replica is replaced at a time, and each new container is started before the old one is stopped, so the service is never taken down during an update. With `monitor: 30s` and `failure_action: rollback`, each new replica is watched for 30 seconds, and the previous version is restored automatically if the new one is found unhealthy.

Resource limits. A CPU and memory limit is set for every service (for example, 0.5 CPU and 256 MB for each core replica). In this way, the rest of the node cannot be starved by a buggy service, and one of the outage causes listed by Schmidt (2006) is addressed: a system being used beyond its design limits.

Services that are not scaled. DBMS and cron jobs are allowed by the assignment to be kept at one replica. Postgres is kept at one because its data files cannot be shared between copies of a plain Postgres. The backup job is kept at one because it is treated as a cron job. Redis is also kept at one, since a single shared cache is sufficient and the database is used as a fallback if Redis is lost.

Deployment steps. Apart from the creation of the secret file, only three commands are needed for deployment:

```
docker compose -f production.yml build
docker compose -f production.yml push
docker stack deploy -c production.yml app
```

Tests and results

`production.yml` was deployed on a single node Swarm on a laptop, with a local registry service and a folder used in place of the NFS share. All six services were brought to their target replicas within about a minute.

```
berkgonenc@Berks-MacBook-Pro app % docker service scale app_core=3

app_core scaled to 3
overall progress: 3 out of 3 tasks
1/3: running   [==================================================>]
2/3: running   [==================================================>]
3/3: running   [==================================================>]
verify: Service app_core converged
berkgonenc@Berks-MacBook-Pro app % docker stack services app
ID             NAME         MODE         REPLICAS   IMAGE                                  PORTS
r2q0g5n2kvn7   app_auto     replicated   1/1        localhost:5000/ccds/auto-feeder:1
la4iwcs44ijf   app_backup   replicated   1/1        localhost:5000/ccds/db-backup:1
pl0dxapfeqmm   app_core     replicated   3/3        localhost:5000/ccds/core-app:2
d5o68de9r82d   app_db       replicated   1/1        postgres:17
ym9uxu427asd   app_redis    replicated   1/1        redis:7-alpine
v2osonl93s5d   app_static   replicated   2/2        localhost:5000/ccds/static:1
berkgonenc@Berks-MacBook-Pro app %
```

Figure 1. `docker stack services app`: every service at its target replica count.

Test	Action taken	Result
Load balancing	`app_core:8000/` was called six times from a container on the proxy network	The requests were answered by three replicas in turn: `df89f6c4ab31`, `20d60e8fdd1c` and `7e1d9f242385`
Self healing	One core container was killed with `docker rm -f`	The container was marked Failed (exit 137), and a replacement was already being started 4 seconds later
Rolling update	`docker service update --force app_core` was run	All three replicas were replaced one by one, and the service was reported as converged
Backup and restore	The ratings table was dropped and `restore.sh` was run	The data was restored from the last dump (see the RPO discussion above)
Health under failure	Postgres and Redis were stopped while the app was running	A 200 was still returned by `/health` in 8 ms, and both were reported as down

```
{"replica":"df89f6c4ab31","service":"core-app"}
{"replica":"20d60e8fdd1c","service":"core-app"}
{"replica":"7e1d9f242385","service":"core-app"}
{"replica":"df89f6c4ab31","service":"core-app"}
{"replica":"20d60e8fdd1c","service":"core-app"}
{"replica":"7e1d9f242385","service":"core-app"}
berkgonenc@Berks-MacBook-Pro app %
```

Figure 2. Load balancing: six requests are answered in turn by replicas df89f6c4ab31, 20d60e8fdd1c and 7e1d9f242385.

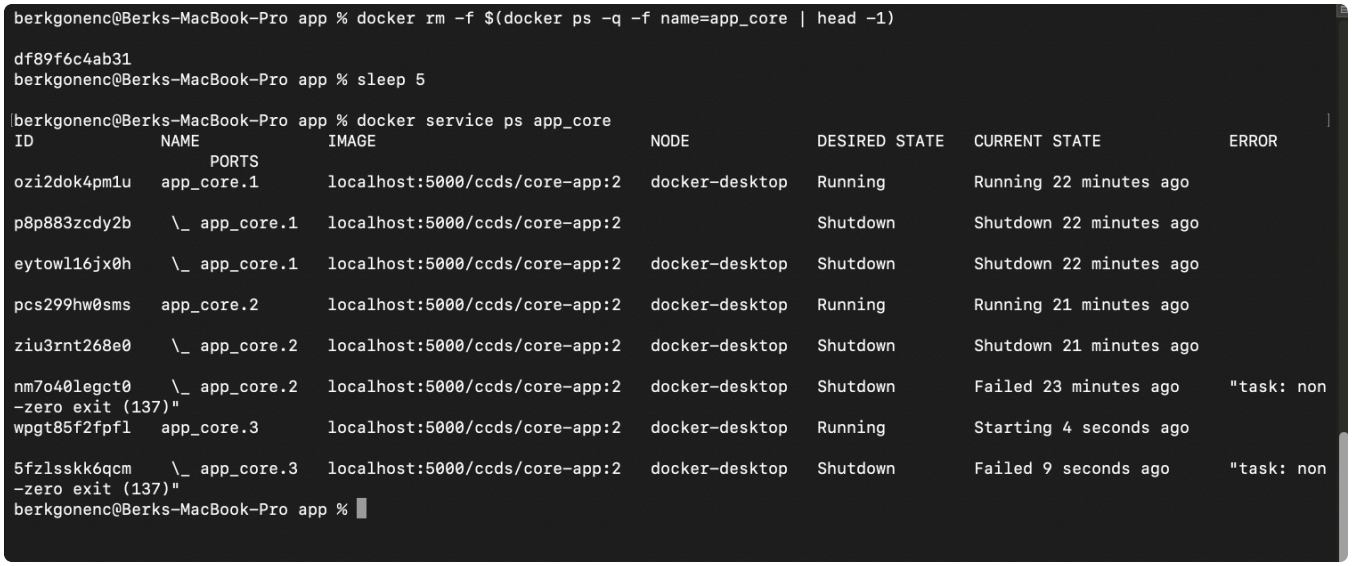


Figure 3. Self healing: replica df89f6c4ab31 is killed (exit 137), and a replacement is already being started by Swarm 4 seconds later.

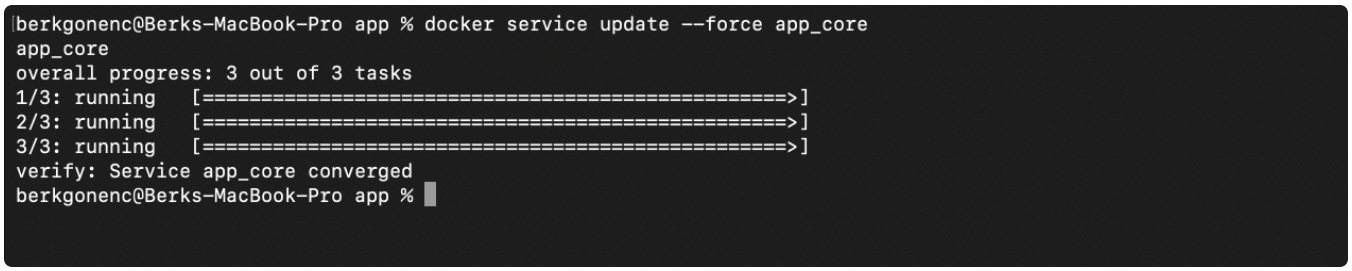


Figure 4. Rolling update: all three replicas are replaced one at a time, and the service is reported as converged.

An unexpected lesson was learned from the rolling update test. On a single node, the update was stalled with the message "no suitable node (max replicas per node limit exceed)". Room for one extra container is needed by the `start-first` order during the update, but only two replicas per node are allowed by the placement rule, and only one node was available on the laptop. The limit was lifted for the local test, and the update was completed. On the three node Swarm, six slots are provided for three replicas, so enough room is available for the update.

Two bugs were caught by the restore drill on Swarm. When the drill was repeated on the Swarm deployment, the restore was rejected with "password authentication failed". Two problems were found as the cause:

1. **The old password was kept by the database.** The data volume from the earlier local run was reused by Swarm, and `POSTGRES_PASSWORD` is only applied by Postgres when a new database is created. As a result, the new Docker secret was ignored, and every

login made with the secret was rejected. The password of an existing database is not changed when a secret is changed; it has to be changed inside Postgres as well.

2. **An empty backup was written and reported as a success.** In `pg_dump | gzip`, only the exit code of gzip is checked by the shell, and nothing was compressed by gzip without any error. A 20 byte file was produced that looked like a valid backup, and it was picked as the newest dump by `restore.sh`.

```
berkgonenc@Berks-MacBook-Pro app % ls -lh ~/ccds-nfs/app-backups
total 8
-rw-r--r--  1 berkgonenc  staff    20B 11 Oct 17:30 ratings-20261011-153058.sql.gz
berkgonenc@Berks-MacBook-Pro app %
```

Figure 5. The bug: a 20 byte "backup" in which no data was stored.

```
berkgonenc@Berks-MacBook-Pro app % docker exec $(docker ps -q -f name=app_db) psql -U ratings -c 'DROP TABLE ratings;'

DROP TABLE
berkgonenc@Berks-MacBook-Pro app % time docker exec $(docker ps -q -f name=app_backup) restore.sh

Restoring /backups/ratings-20261011-153058.sql.gz into ratings on db
psql: error: connection to server at "db" (10.0.2.2), port 5432 failed: FATAL:  password authentication failed for user "ratings"
docker exec $(docker ps -q -f name=app_backup) restore.sh  0.02s user 0.01s system 19% cpu 0.146 total
berkgonenc@Berks-MacBook-Pro app % docker run --rm --network proxy curlimages/curl -s app_core:8000/leaderboard
<!doctype html>
<html lang=en>
<title>500 Internal Server Error</title>
<h1>Internal Server Error</h1>
<p>The server encountered an internal error and was unable to complete your request. Either the server is overloaded or there is an er
ror in the application.</p>
berkgonenc@Berks-MacBook-Pro app %
```

Figure 6. The failed restore: the password is rejected, and an HTML 500 page is returned by the app.

The backup job was fixed so that the output is compressed by `pg_dump` itself (its exit code now decides success), and each dump is checked for the end marker written by `pg_dump` at the bottom of every complete dump. `restore.sh` was fixed so that incomplete dumps are skipped, the newest valid dump is used instead, and the database login is tested before anything is dropped. The app was also changed so that a clear 503 "database unavailable" is returned instead of an HTML 500 page. Without this drill, the empty backups would only have been discovered during a real disaster.

After the fix, the old volume was wiped, the stack was redeployed, and the drill was run again.

```
berkgonenc@Berks-MacBook-Pro app % docker service update --force app_backup

app_backup
overall progress: 1 out of 1 tasks
1/1: running   [==================================================>]
verify: Service app_backup converged
berkgonenc@Berks-MacBook-Pro app % sleep 5
berkgonenc@Berks-MacBook-Pro app % ls -lh ~/ccds-nfs/app-backups
total 8
-rw-r--r--  1 berkgonenc  staff   1.2K 11 Oct 18:09 ratings-20261011-160946.sql.gz
berkgonenc@Berks-MacBook-Pro app %
```

Figure 7. A real 1.2 KB dump is now written to the NFS folder by a forced backup.

```
berkgonenc@Berks-MacBook-Pro app % docker exec $(docker ps -q -f name=app_db) psql -U ratings -c 'DROP TABLE ratings;'

DROP TABLE
berkgonenc@Berks-MacBook-Pro app % time docker exec $(docker ps -q -f name=app_backup) restore.sh

Restoring /backups/ratings-20261011-160946.sql.gz into ratings on db
Restore finished
docker exec $(docker ps -q -f name=app_backup) restore.sh  0.01s user 0.01s system 5% cpu 0.383 total
berkgonenc@Berks-MacBook-Pro app % docker run --rm --network proxy curlimages/curl -s app_core:8000/leaderboard
[{"avg":5.0,"page":"github.com","votes":3},{"avg":4.5,"page":"docs.docker.com","votes":2},{"avg":3.0,"page":"atenea.upc.edu","votes":2
},{"avg":2.33,"page":"upc.edu","votes":3},{"avg":1.75,"page":"wikipedia.org","votes":4}]
berkgonenc@Berks-MacBook-Pro app %
```

Figure 8. The repeated drill: the table is dropped, the restore is completed in 0.383 seconds, and all five pages are returned by the leaderboard.

Limitations and lessons learned

- **The database is a single point of failure.** Only one Postgres replica is run on one node, so writes are stopped by a node failure until a restore is completed. A real fix would require Postgres replication (a primary and a standby), which is beyond the scope of this lab.
- **The RPO is set to one hour.** It could be lowered with more frequent dumps or with continuous WAL archiving, at the cost of more storage and load.
- **Spare capacity is needed for zero downtime updates.** It was shown by the stalled rolling update that one free slot per node is required by `start-first`. Rules that are designed to protect availability can be blocked by each other when the cluster is too small.
- **Health checks have to be matched to what can be fixed by a restart.** When a health check depends on another service, an outage of that service is turned into a restart storm. Now only liveness is checked, and dependencies are reported separately.
- **Startup is only ordered by `depends_on`.** The app is not held back until connections are accepted by Postgres, so the connection is retried for about 30 seconds at startup instead of the app being crashed.

How to run

Locally (from the `app/` folder):

```
docker compose up -d --build
curl localhost:8000/leaderboard
```

On the Swarm. The following items must already be provided by the basic stack: the attachable `proxy` overlay network, the NFS share mounted at `/mnt/nfs` on every node with a writable `app-backups/` folder, and one node labelled `db=true`. Then the following commands are run:

```
cp secrets/db_password.txt.example secrets/db_password.txt
openssl rand -base64 24 > secrets/db_password.txt
docker compose -f production.yml build
docker compose -f production.yml push
docker stack deploy -c production.yml app
```

Disaster recovery drill:

```
docker exec -it $(docker ps -q -f name=app_backup) restore.sh
```

References

- Schmidt, K. (2006). *High availability and disaster recovery. Concepts, design, implementation.* Springer.
- Poulton, N. (2020). *Docker deep dive. Zero to Docker in a single book.* 2nd ed. Packt.